

Introduction to the Rust Programming Language

2026-10-01 — A Purdue Hackers workshop

Arhan Chaudhary

<https://github.com/ArhanChaudhary>

arhan.ch@gmail.com



Ferris, the Rust mascot

What is Rust?

What is Rust?

What is Rust?

Rust is a general-purpose programming language that empowers everyone to build reliable and efficient software.

— <https://rust-lang.org/>

What is Rust?

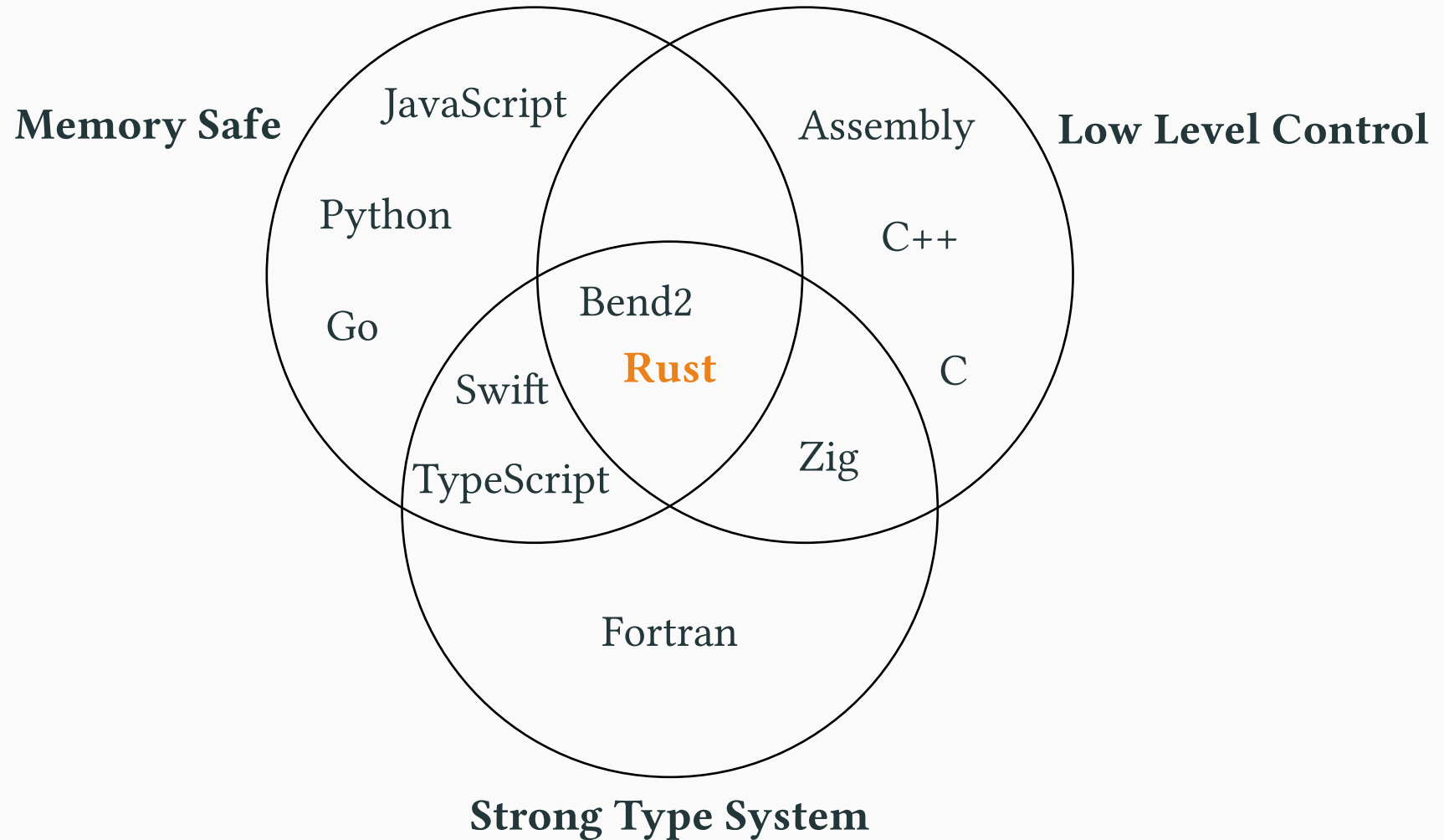
Rust is a general-purpose programming language that empowers everyone to build reliable and efficient software.

— <https://rust-lang.org/>

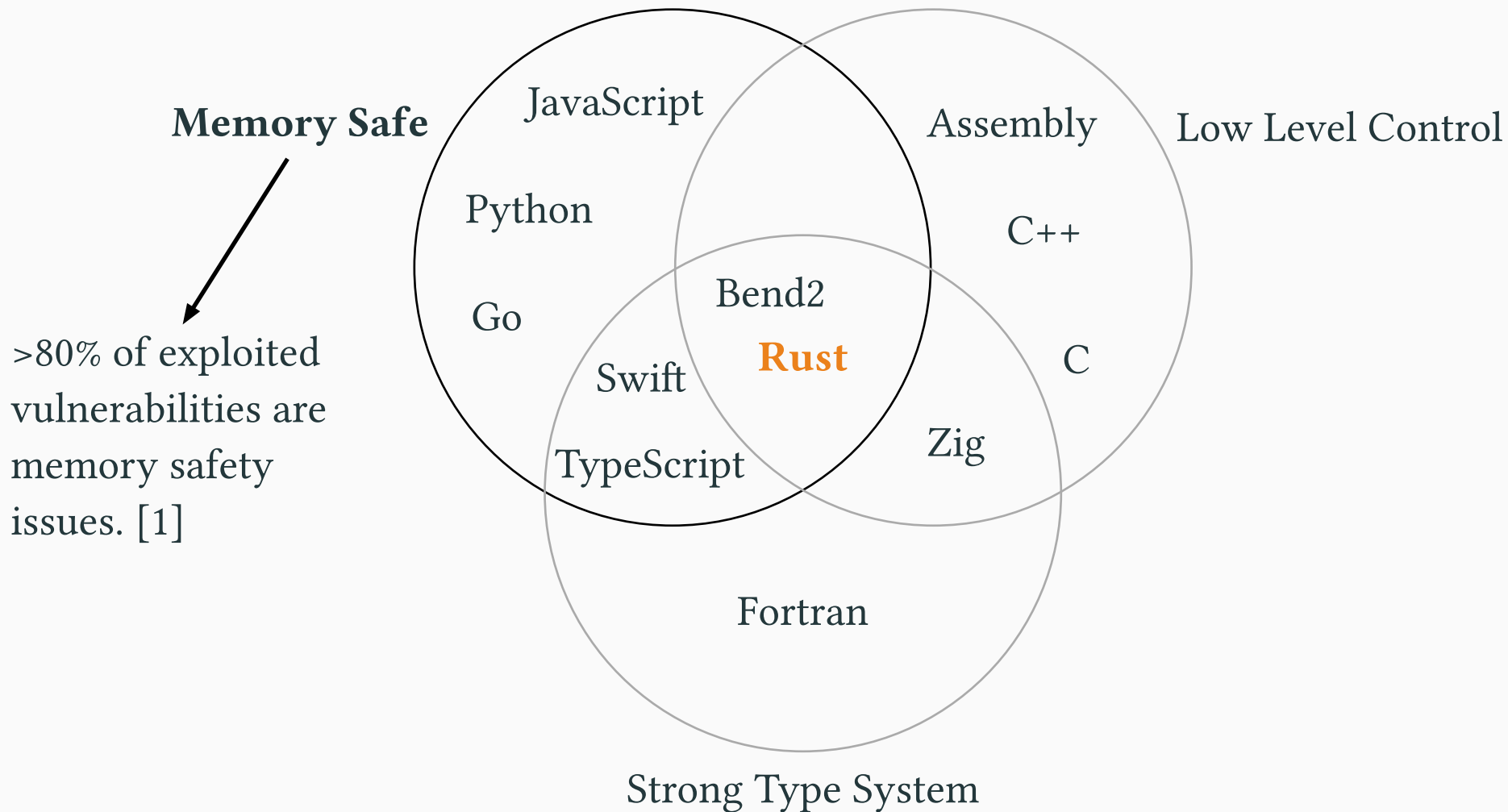
How?



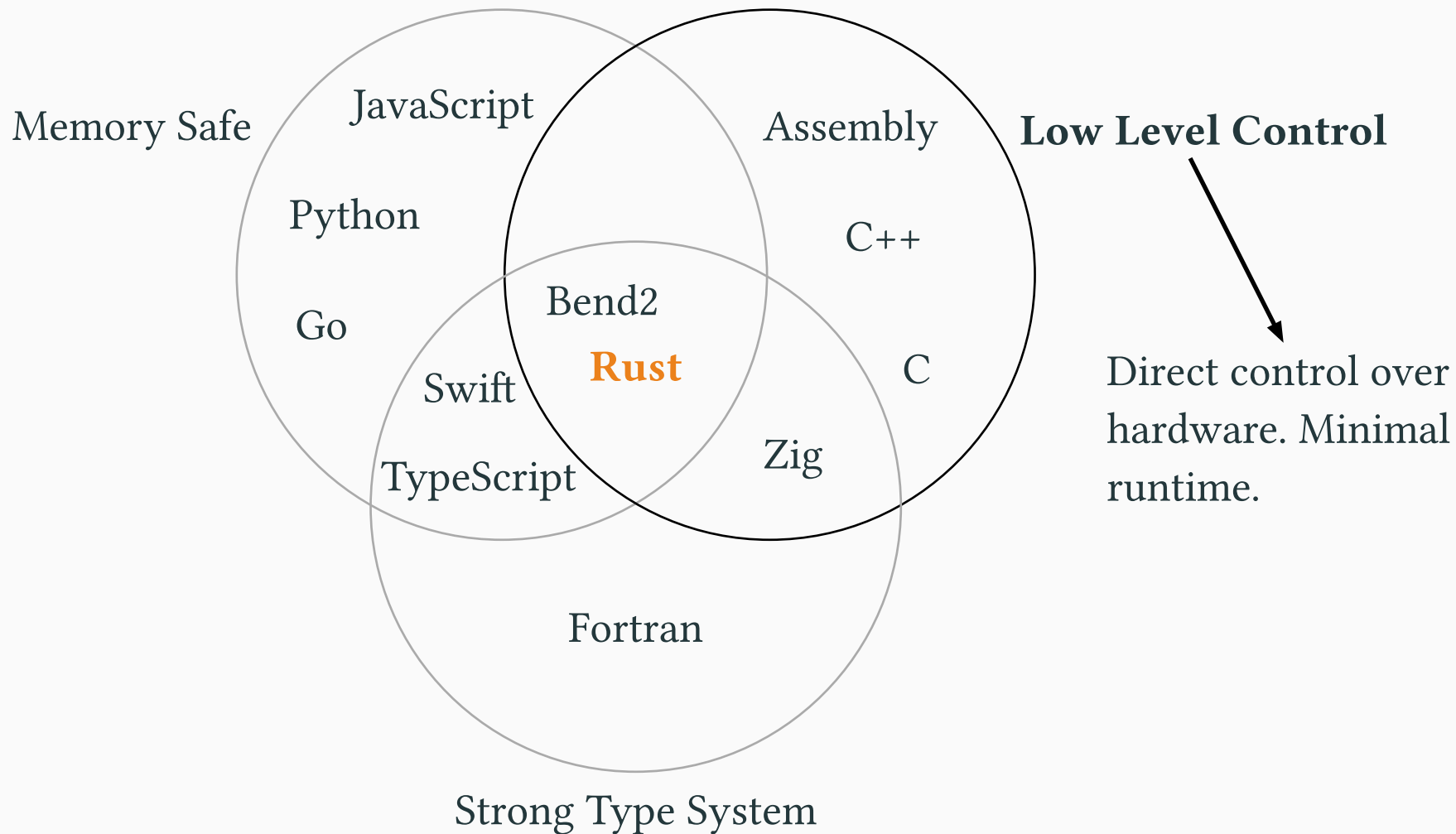
How is Rust empowering?



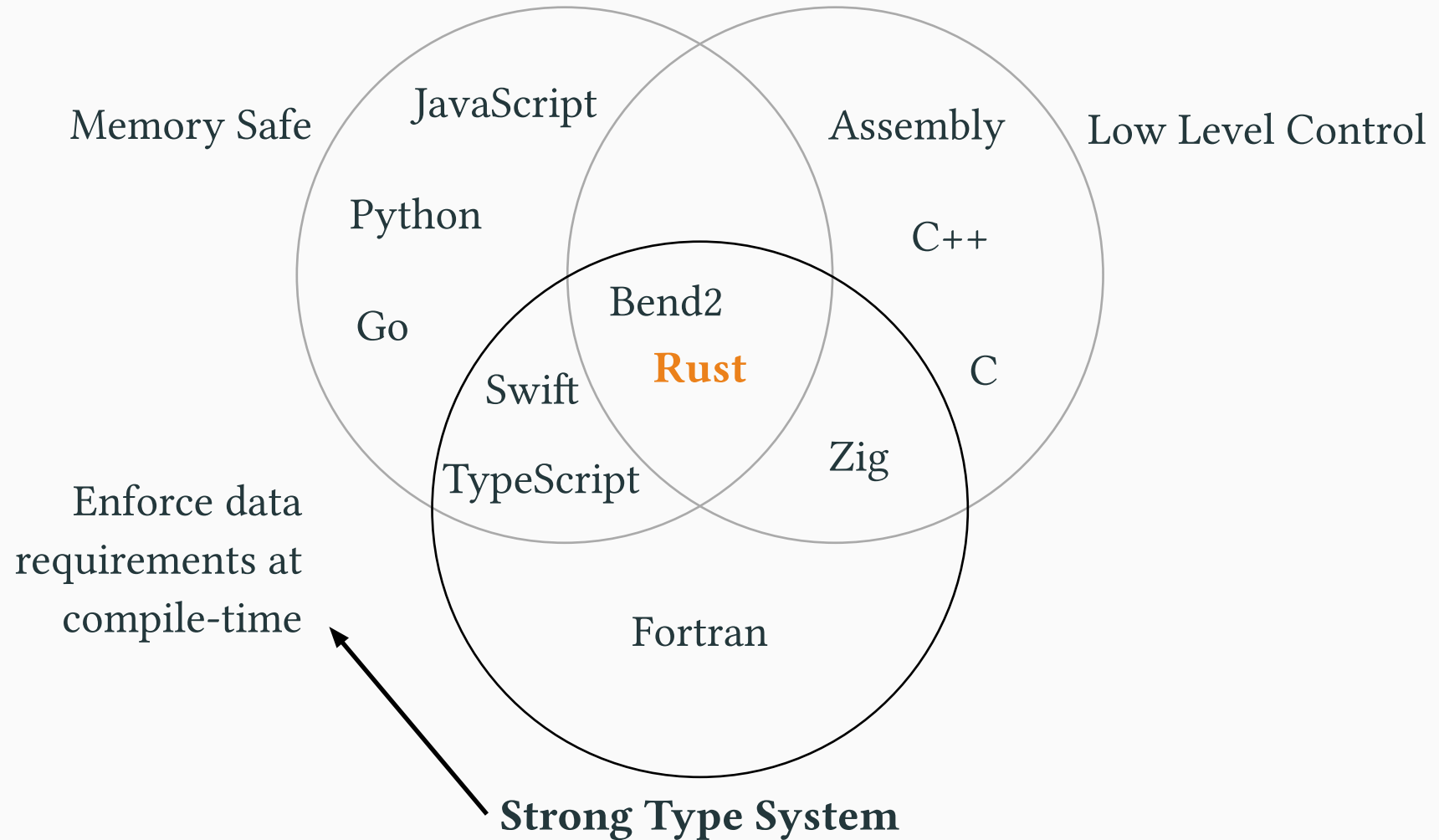
How is Rust empowering?



How is Rust empowering?

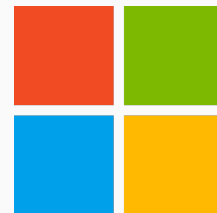
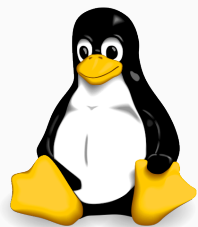


How is Rust empowering?



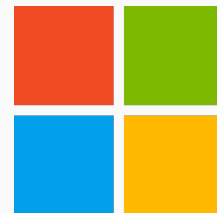
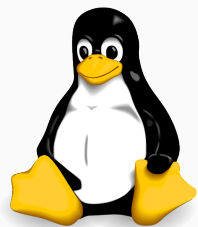
Who uses Rust?

You probably run Rust code every day



Who uses Rust?

You probably run Rust code every day



As do AI agents

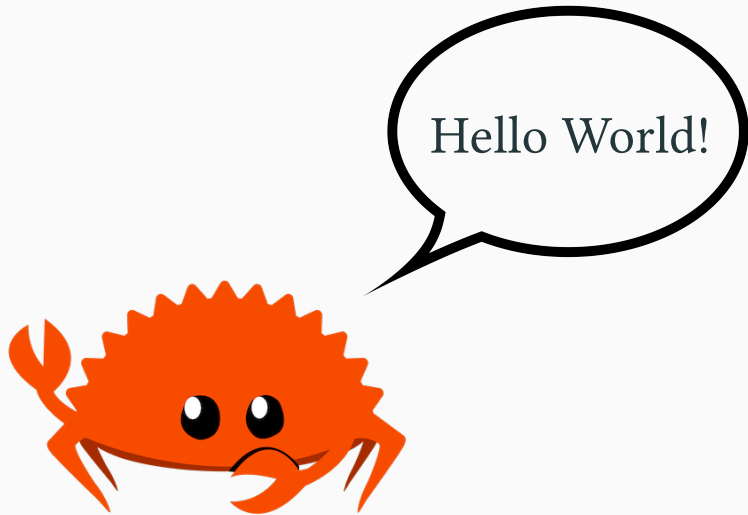


Let's learn some Rust!

<https://code.purduehackers.com/new/purduehackers/rust-workshop-2026>

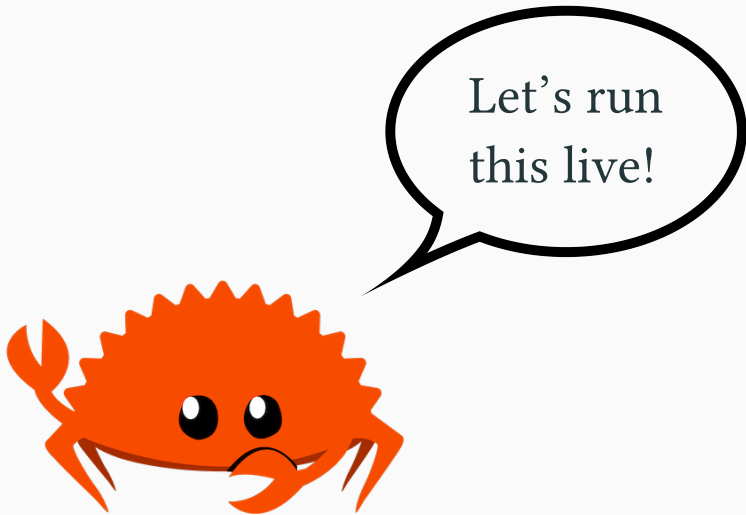
Hello World

```
fn main() {  
    println!("Hello World!");  
}
```



Hello World

```
fn main() {  
    println!("Hello World!");  
}
```



Guessing Game

```
$ cargo run --bin guessing-game
   Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.10s
   Running `target/debug/guessing-game`
Guess the number!
Please input your guess.
60
You guessed: 60
Too small!
Please input your guess.
80
You guessed: 80
Too big!
Please input your guess.
73
You guessed: 73
Too big!
```

```
Please input your guess.
66
You guessed: 66
Too small!
Please input your guess.
70
You guessed: 70
Too big!
Please input your guess.
68
You guessed: 68
You win!
```

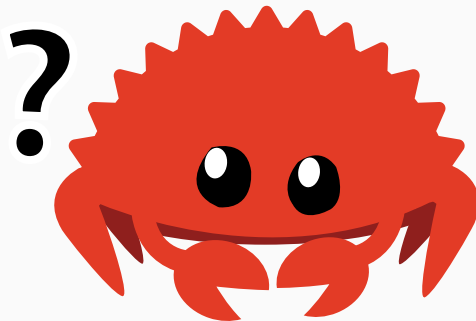
Variables

```
fn mutability() {  
    let power = 1000;  
    println!("The power level is: {power}");  
    power = 2000;  
    println!("The power level is now: {power}");  
}
```

Variables

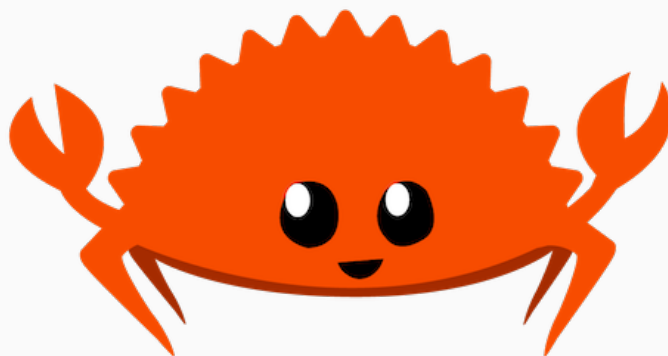
```
fn mutability() {  
    let power = 1000;  
    println!("The power level is: {power}");  
    power = 2000;  
    println!("The power level is now: {power}");  
}
```

error[E0384]: cannot assign twice to immutable variable power



Variables

```
fn mutability() {  
    let power = 1000;  
    let mut power = 1000;  
    println!("The power level is: {power}");  
    power = 2000;  
    println!("The power level is now: {power}");  
}
```



Variables

```
fn scope() {  
    let power = 5000;  
    println!("The power level is: {power}");  
  
}
```

Variables

```
fn scope() {  
    let power = 5000;  
    println!("The power level is: {power}");  
  
    {  
        let power = power * 2; // 10000  
        println!("The power level is: {power}. The power level  
is over 9000!!");  
    }  
  
}
```

Variables

```
fn scope() {  
    let power = 5000;  
    println!("The power level is: {power}");  
  
    {  
        let power = power * 2; // 10000  
        println!("The power level is: {power}. The power level  
is over 9000!!");  
    }  
  
    println!("The power level is back to: {power}");  
}
```

Variables

```
fn scope() {  
    let power = 5000;  
    println!("The power level is: {power}");  
  
    { // this creates a scope  
        let power = power * 2; // 10000  
        println!("The power level is: {power}. The power level  
is over 9000!!");  
    }  
  
    println!("The power level is back to: {power}");  
}
```

Variables

```
fn data_types() {  
    // Every variable has a singular, fixed type!  
    // `int` has the integer type `i32`  
    let mut int = 5;  
  
}
```

Variables

```
fn data_types() {  
    // Every variable has a singular, fixed type!  
    // `int` has the integer type `i32`  
    let mut int = 5;  
    // `int2` is optionally annotated explicitly  
    let int2: i32 = 5;  
  
}
```

Variables

```
fn data_types() {  
    // Every variable has a singular, fixed type!  
    // `int` has the integer type `i32`  
    let mut int = 5;  
    // `int2` is optionally annotated explicitly  
    let int2: i32 = 5;  
    let true_or_false: bool = true;  
  
}
```

Variables

```
fn data_types() {  
    // Every variable has a singular, fixed type!  
    // `int` has the integer type `i32`  
    let mut int = 5;  
    // `int2` is optionally annotated explicitly  
    let int2: i32 = 5;  
    let true_or_false: bool = true;  
    let decimal: f64 = 4.5;  
  
}
```

Variables

```
fn data_types() {  
    // Every variable has a singular, fixed type!  
    // `int` has the integer type `i32`  
    let mut int = 5;  
    // `int2` is optionally annotated explicitly  
    let int2: i32 = 5;  
    let true_or_false: bool = true;  
    let decimal: f64 = 4.5;  
  
    int = decimal; // different types  
}
```

error[E0308]: mismatched types

```
fn main() {  
    // Reads a user input number from the terminal  
    let int: i32 = utils::read_number();  
  
}
```

Control Flow

```
fn main() {  
    // Reads a user input number from the terminal  
    let int: i32 = utils::read_number();  
    if int == 42 {  
        println!("42 is the meaning of life");  
    }  
  
}
```

Control Flow

```
fn main() {  
    // Reads a user input number from the terminal  
    let int: i32 = utils::read_number();  
    if int == 42 {  
        println!("42 is the meaning of life");  
    }  
    let abs = if int < 0 {  
        -int  
    } else {  
        int  
    };  
}
```

Control Flow

```
fn main() {  
    // Reads a user input number from the terminal  
    let int: i32 = utils::read_number();  
  
}
```

Control Flow

```
fn main() {  
    // Reads a user input number from the terminal  
    let int: i32 = utils::read_number();  
    // If `x` is positive, let's  
    // subtract one until `x` is zero.  
    while x != 0 {  
        println!("Count down: {x}");  
        x -= 1;  
    }  
}
```


Ownership can be confusing!
Don't be afraid to ask questions!

How do programming languages manage their memory?

How do programming languages manage their memory?

- Python, Java, Typescript: automatically with overhead

How do programming languages manage their memory?

- Python, Java, Typescript: automatically with overhead
- C, C++: manually, by yourself

Ownership

How do programming languages manage their memory?

- Python, Java, Typescript: automatically with overhead
- C, C++: manually, by yourself

What does Rust do?

Ownership

How do programming languages manage their memory?

- Python, Java, Typescript: automatically with overhead
- C, C++: manually, by yourself

What does Rust do?

It picks a third option:

- Rust: the compiler enforces the rules of ownership!

Ownership

How do programming languages manage their memory?

- Python, Java, Typescript: automatically with overhead
- C, C++: manually, by yourself

What does Rust do?

It picks a third option:

- Rust: the compiler enforces the rules of ownership!
 - The rules manage memory automatically **without overhead**

Ownership

How do programming languages manage their memory?

- Python, Java, Typescript: automatically with overhead
- C, C++: manually, by yourself

What does Rust do?

It picks a third option:

- Rust: the compiler enforces the rules of ownership!
 - The rules manage memory automatically **without overhead**
 - Memory unsafe $\Rightarrow$ the rules are violated $\Rightarrow$ your program won't compile

Ownership

Let's focus on a very common data structure: strings

```
fn main() {  
    let ph: &str = "Purdue Hackers";  
    println!("{ph} is awesome!");  
}
```

Ownership

Let's focus on a very common data structure: strings

```
fn main() {  
    let ph: &str = "Purdue Hackers";  
    println!("{ph} is awesome!");  
}
```

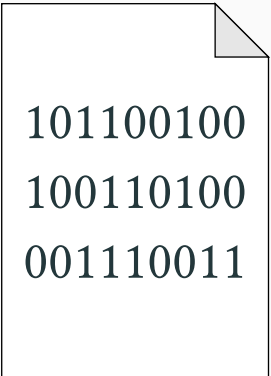
“Purdue Hackers” is hardcoded into the program executable

Ownership

Let's focus on a very common data structure: strings

```
fn main() {  
    let ph: &str = "Purdue Hackers";  
    println!("{ph} is awesome!");  
}
```

“Purdue Hackers” is hardcoded into the program executable



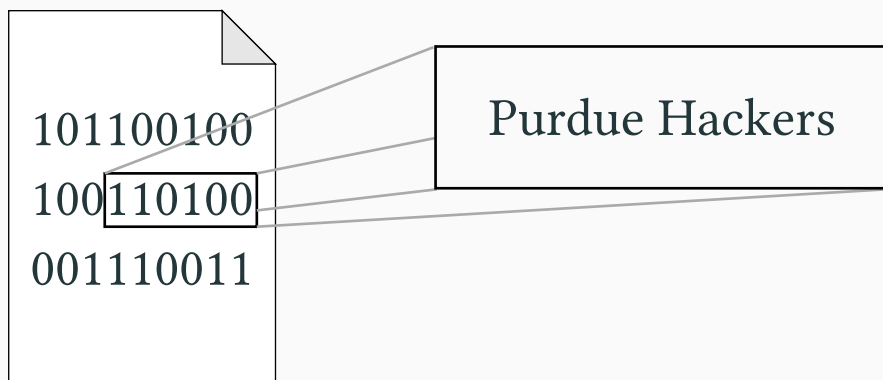
```
101100100  
100110100  
001110011
```

Ownership

Let's focus on a very common data structure: strings

```
fn main() {  
    let ph: &str = "Purdue Hackers";  
    println!("{ph} is awesome!");  
}
```

“Purdue Hackers” is hardcoded into the program executable

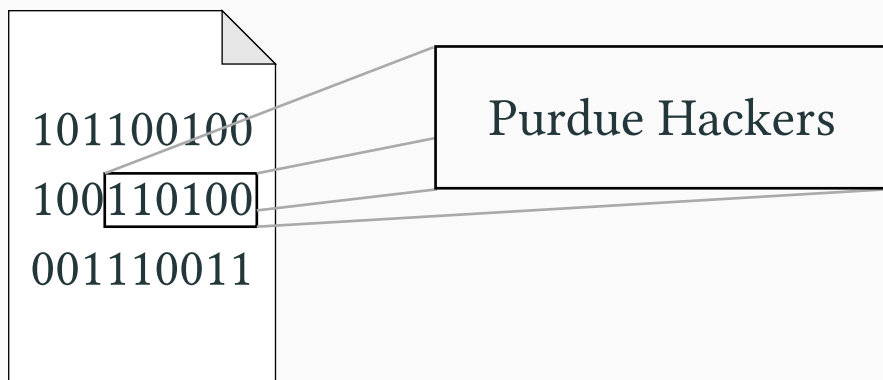


Ownership

Let's focus on a very common data structure: strings

```
fn main() {  
    let ph: &str = "Purdue Hackers";  
    println!("{ph} is awesome!");  
}
```

“Purdue Hackers” is hardcoded into the program executable



Let's prove it!

```
xxd target/debug/ownership | less
```

Ownership

Let's focus on a very common data structure: strings

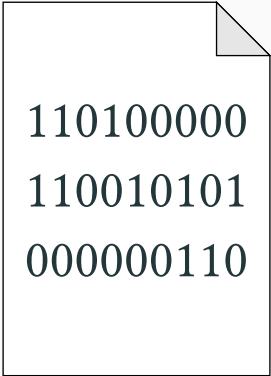
```
fn main() {  
    // Reads user input from the terminal into a "String"  
    let a: String = utils::read_string();  
    println!("{a} is awesome!");  
}
```

Ownership

Let's focus on a very common data structure: strings

```
fn main() {  
    // Reads user input from the terminal into a "String"  
    let a: String = utils::read_string();  
    println!("{a} is awesome!");  
}
```

Input

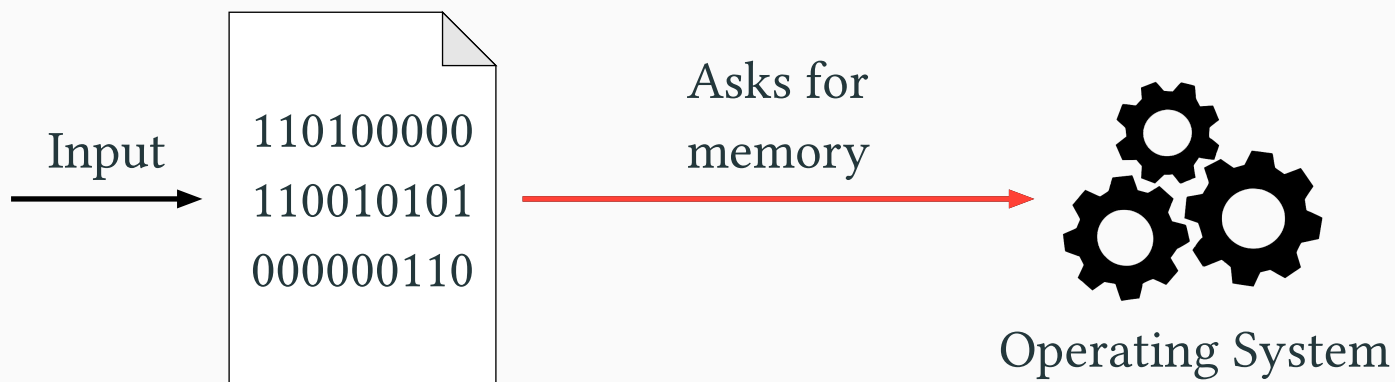
A box with a folded top-right corner containing three lines of binary code.

```
110100000  
110010101  
000000110
```

Ownership

Let's focus on a very common data structure: strings

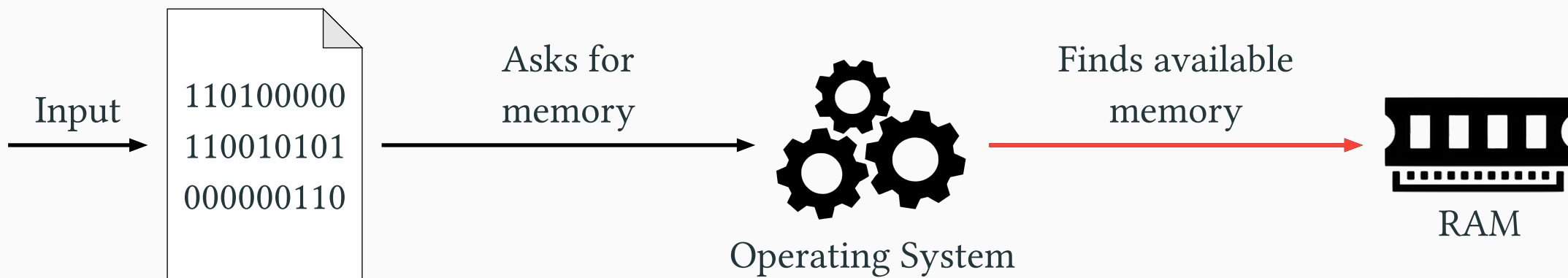
```
fn main() {  
    // Reads user input from the terminal into a "String"  
    let a: String = utils::read_string();  
    println!("{a} is awesome!");  
}
```



Ownership

Let's focus on a very common data structure: strings

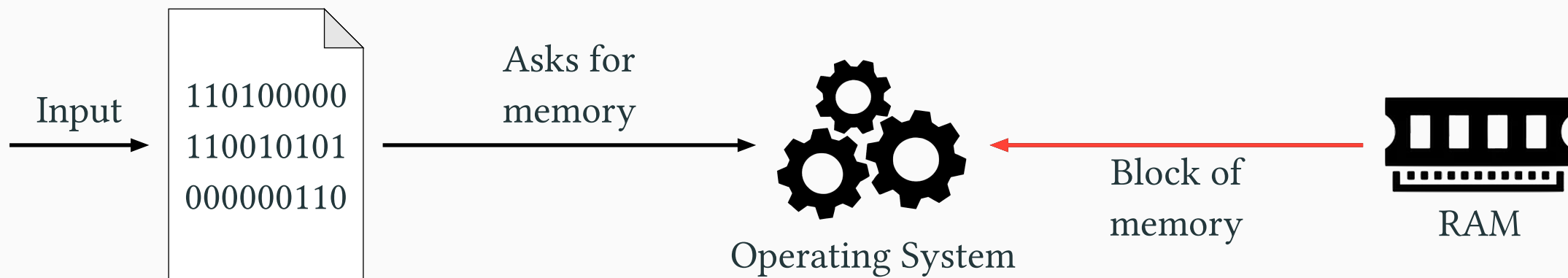
```
fn main() {  
    // Reads user input from the terminal into a "String"  
    let a: String = utils::read_string();  
    println!("{a} is awesome!");  
}
```



Ownership

Let's focus on a very common data structure: strings

```
fn main() {  
    // Reads user input from the terminal into a "String"  
    let a: String = utils::read_string();  
    println!("{a} is awesome!");  
}
```



Ownership

Let's focus on a very common data structure: strings

```
fn main() {  
    // Reads user input from the terminal into a "String"  
    let a: String = utils::read_string();  
    println!("{a} is awesome!");  
}
```



Ownership

Let's focus on a very common data structure: strings

```
fn main() {  
    // Reads user input from the terminal into a "String"  
    let a: String = utils::read_string();  
    println!("{a} is awesome!");  
}
```



Note: extremely simplified

Ownership

```
// Reads user input from the terminal into a "String"  
let a: String = utils::read_string();
```

a

Type	Value
address	0x557593ec3d60
length	5

Memory address	Value
0x557593ec3d60	H
0x557593ec3d61	e
0x557593ec3d62	l
0x557593ec3d63	l
0x557593ec3d64	o

Ownership

```
// Reads user input from the terminal into a "String"  
let a: String = utils::read_string();
```

a

Type	Value
address	0x557593ec3d60
length	5



Memory address	Value
0x557593ec3d60	H
0x557593ec3d61	e
0x557593ec3d62	l
0x557593ec3d63	l
0x557593ec3d64	o

Ownership

```
// Reads user input from the terminal into a "String"  
let a: String = utils::read_string();
```

a

Type	Value
address	0x557593ec3d60
length	5



Value
H
e
l
l
o

Ownership

How do Strings interact with scopes?

```
fn main() {  
    {  
        // `a` is created inside a scope  
        let a: String = utils::read_string();  
        // `a` is used inside a scope  
        println!("{a} is awesome!");  
    }  
    // `a` has left its scope  
}
```

Ownership

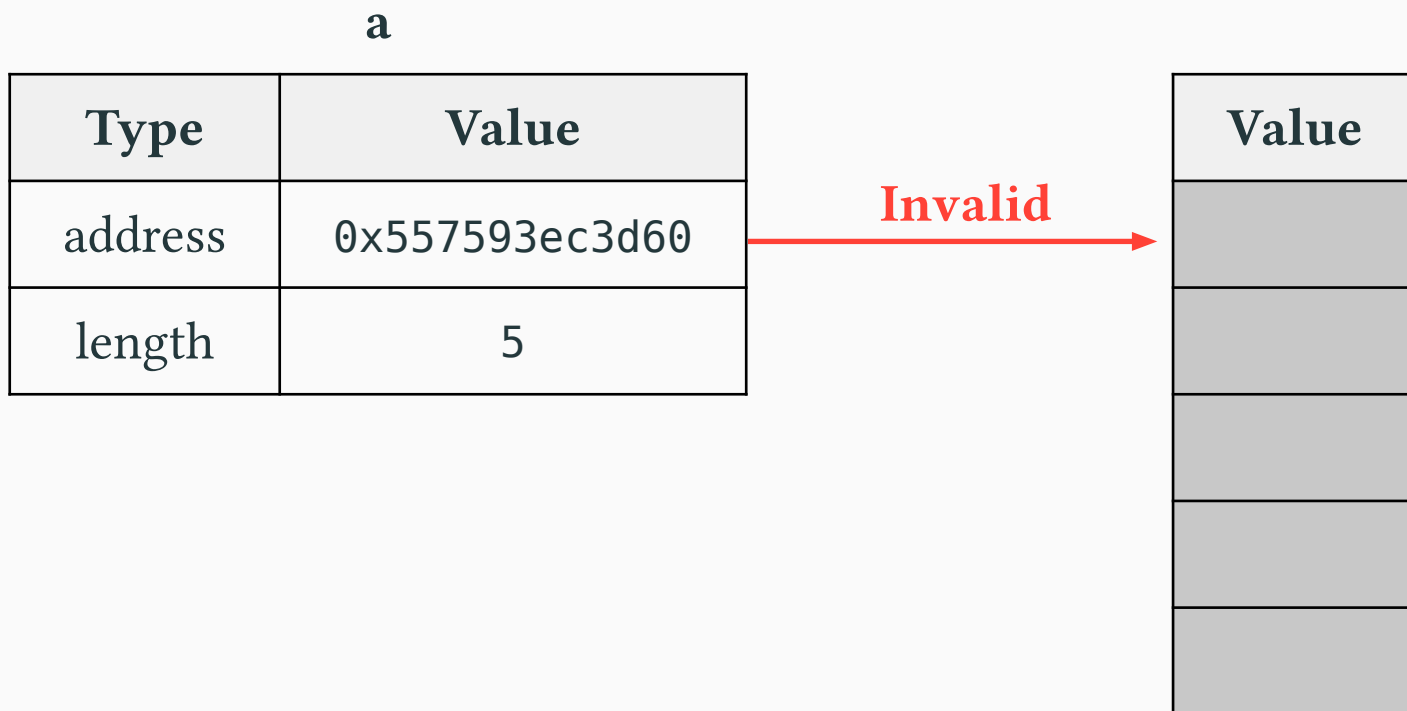
How do Strings interact with scopes?

```
fn main() {  
    {  
        // `a` is created inside a scope  
        let a: String = utils::read_string();  
        // `a` is used inside a scope  
        println!("{a} is awesome!");  
    }  
    // `a` has left its scope  
    // What happened to the memory?  
}
```

Ownership

Remember: Rust manages memory automatically

- Rust inserts code to free up memory back to the operating system



Ownership

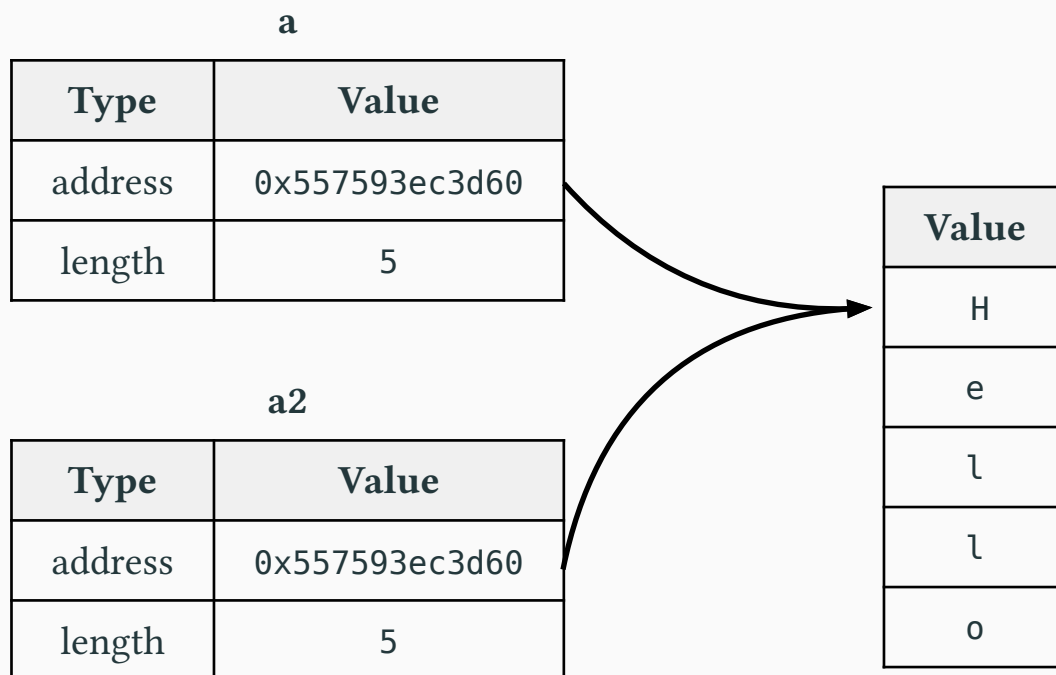
```
fn main() {  
    {  
        let a: String = utils::read_string();  
        let a2: String = a;  
        println!("{a} is awesome!");  
    }  
}
```

Ownership

```
fn main() {  
    let a: String = utils::read_string();  
    let a2: String = a;  
    println!("{a} is awesome!");  
}
```

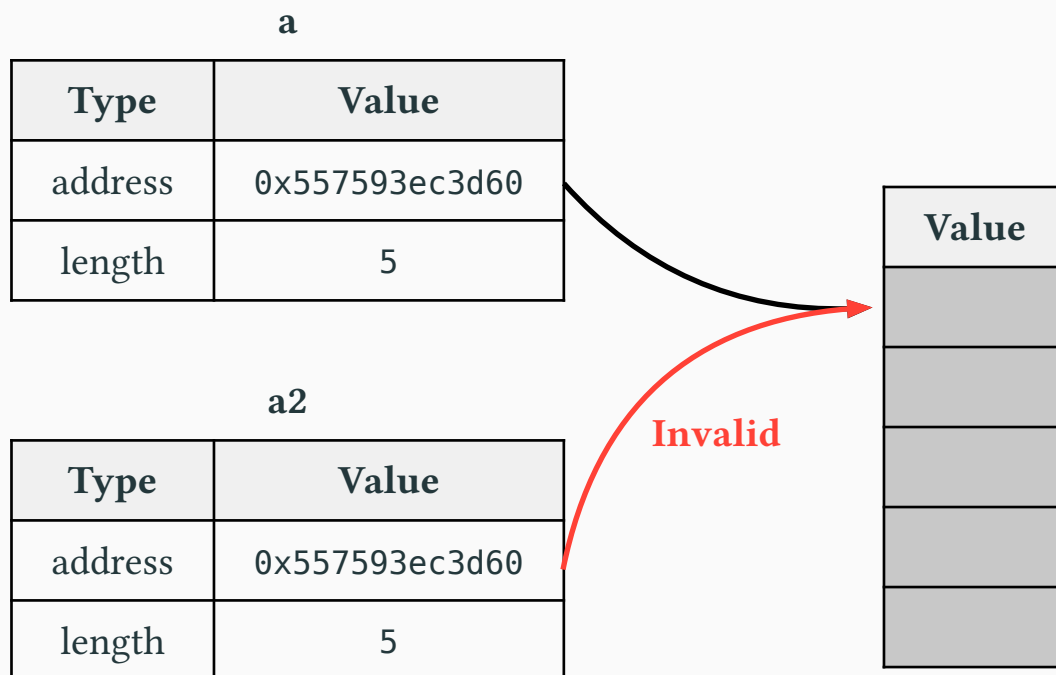
Ownership

```
fn main() {  
    let a: String = utils::read_string();  
    let a2: String = a;  
    println!("{a} is awesome!");  
}
```



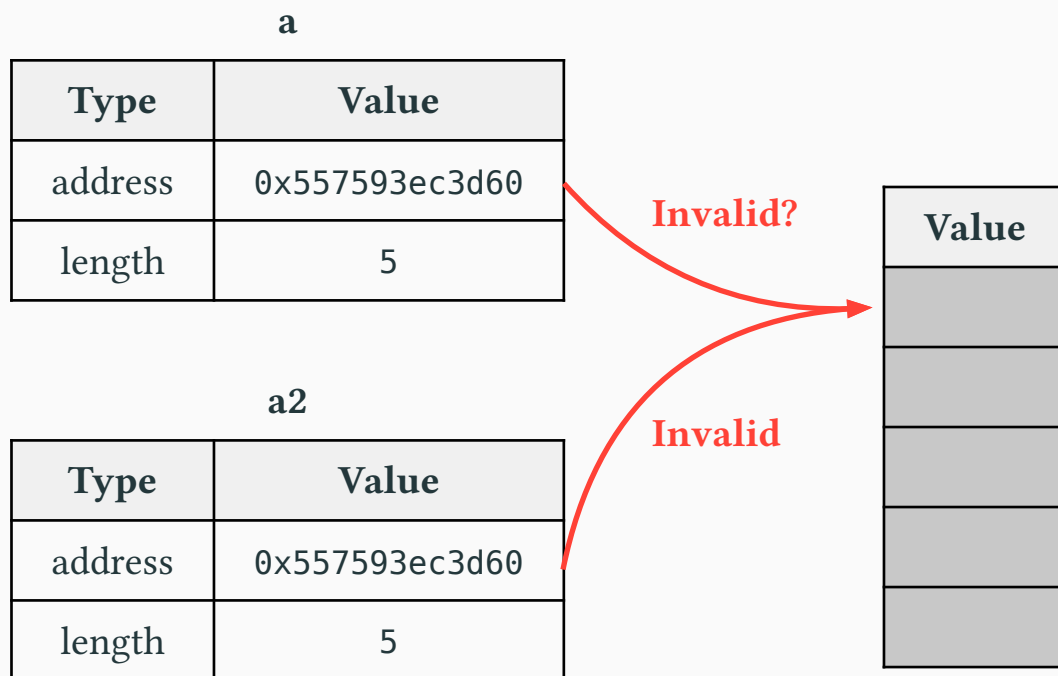
Ownership

```
fn main() {  
    let a: String = utils::read_string();  
    let a2: String = a;  
    println!("{a} is awesome!");  
}
```



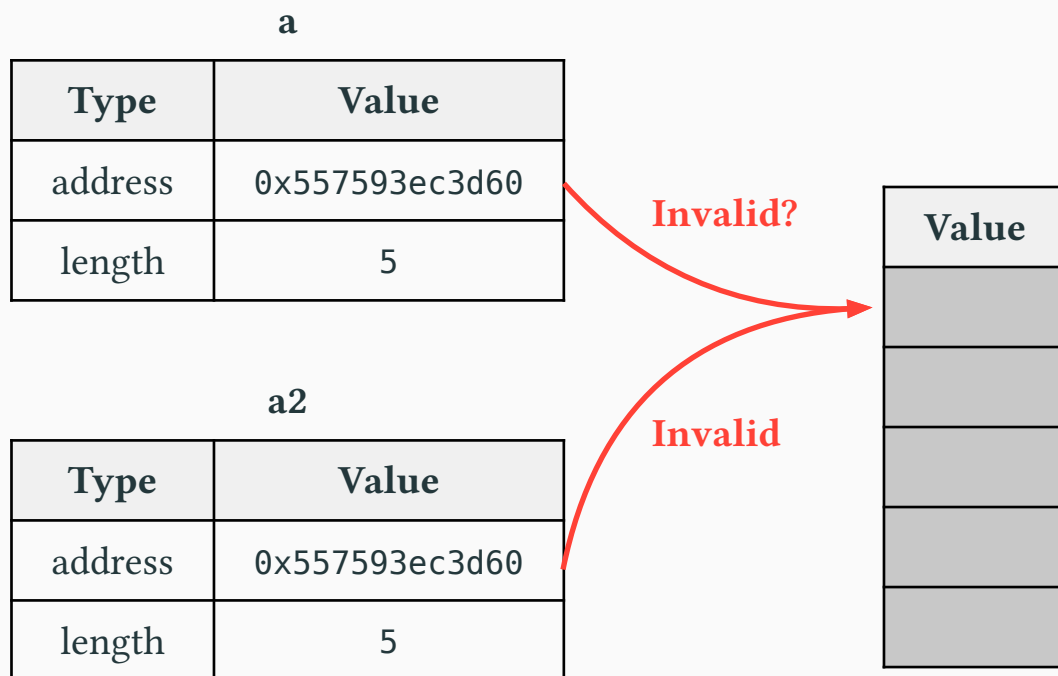
Ownership

```
fn main() {  
    let a: String = utils::read_string();  
    let a2: String = a;  
    println!("{a} is awesome!");  
}
```



Ownership

```
fn main() {  
    let a: String = utils::read_string();  
    let a2: String = a;  
    println!("{a} is awesome!");  
}
```

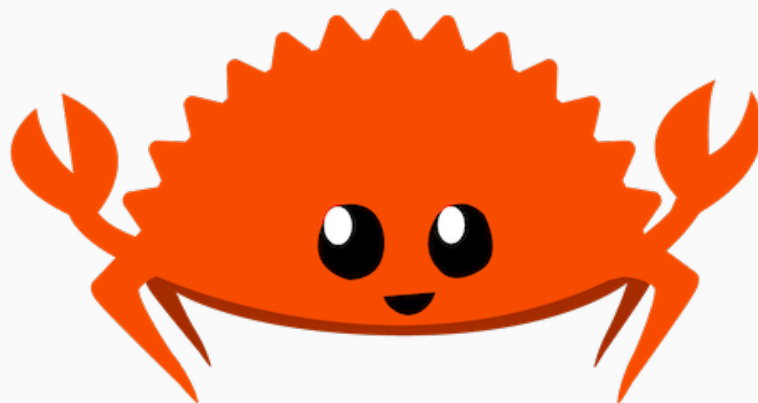


This is a memory safety bug!
This leads to undefined behavior!

Ownership

```
fn main() {  
    let a: String = utils::read_string();  
    let a2: String = a;  
    println!("{a} is awesome!");  
}
```

error[E0382]: borrow of moved value: `a`



What are the rules of ownership?

Ownership

What are the rules of ownership?

- Each value in Rust has an owner

What are the rules of ownership?

- Each value in Rust has an owner
- There can only be one owner at a time

What are the rules of ownership?

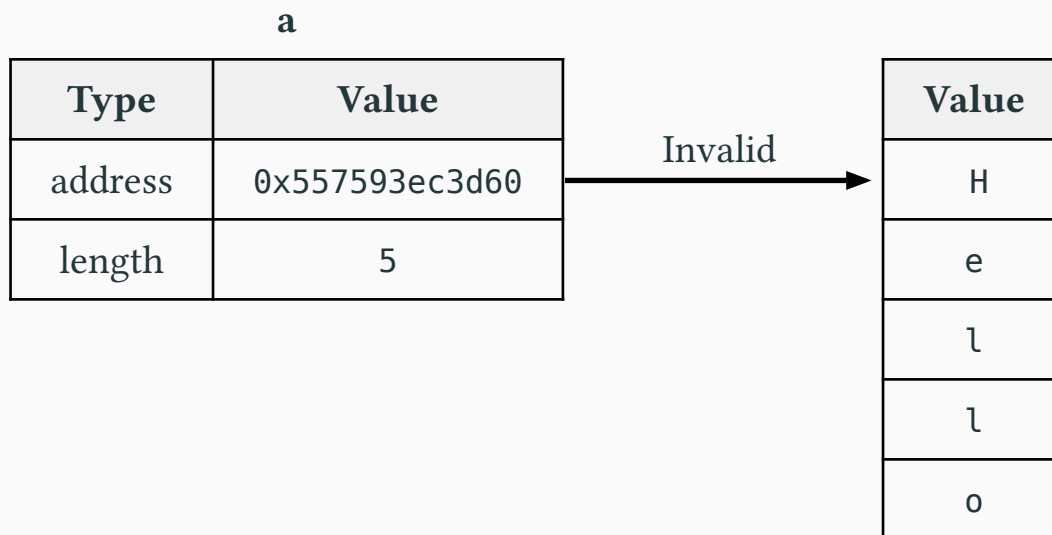
- Each value in Rust has an owner
- There can only be one owner at a time
- When the owner goes out of scope, the value will be cleaned up

Ownership

```
fn main() {  
    let a: String = utils::read_string();  
    let a2: String = a;  
    println!("{a} is awesome!");  
}
```

Ownership

```
fn main() {  
    let a: String = utils::read_string();  
    let a2: String = a;  
    let a2: String = a.clone();  
    println!("{a} is awesome!");  
}
```

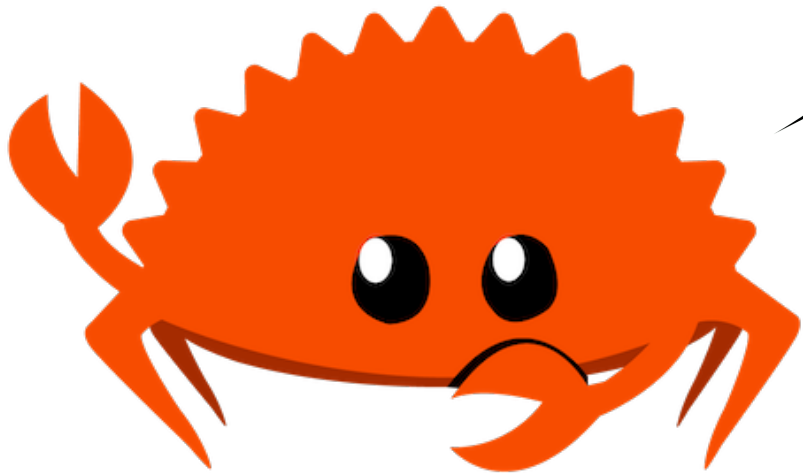


Ownership

```
fn main() {  
    let a: String = utils::read_string();  
    let a2: String = a;  
    println!("{a} is awesome!");  
    println!("{a2} is awesome!");  
}
```

Further Reading

- [The Rust Programming Language](#)
- [Rustlings](#)



Thank you
for listening!

Bibliography

- [1] “What is memory safety and why does it matter?,” *Internet Security Research Group*. [Online]. Available: <https://www.memorysafety.org/docs/memory-safety/>